

Library **Management** System

ITP 2 Final Defense

Developed by:

Alikhan Smagul | Ibrai Muratbek | Sultanali Baltabay | Rizabek Begen

Problem Description

Tracking library books manually or using simple unoptimized scripts is slow and error-prone.

- ⚠ **Data Volatility:** Without saving data to a file, all records disappear when the program closes.
- ⌚ **O(n) Bottlenecks:** Searching through standard arrays causes severe performance drops as databases grow.
- 💥 **Unhandled Logic:** Without Guard Clauses, programs crash when users input invalid data or attempt to duplicate borrows.



System Architecture



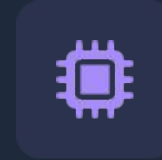
Storage

Direct reading and writing of books.json and users.json ensuring complete data persistence.



Models

Strict Object-Oriented classes (Book, User, Admin) handling encapsulated states and inheritance.



Service Layer

The core logic engine running $O(1)$ dictionary lookups, generators, and transactional validations.

Data Models & OOP (Alikhan)

Encapsulation & Inheritance

 **Encapsulation:** We strictly protect data integrity using private variables (__book_id).

 **Inheritance:** The Admin class logically extends the base User class.

```
class Book:
    def __init__(self, book_id, title):
        self.__book_id = book_id
        self.__title = title
        self.__available = True

class Admin(User):
    def __init__(self, user_id, name):
        super().__init__(user_id, name)
        self.role = "admin"
```

Storage & Utils (Ibrai)

Dynamic JSON Parsing

-  **Serialization:** Safely converting Python instances to dicts for JSON dumping.
-  **Role Identification:** During deserialization, the file service dynamically allocates either a User or Admin.

```
def load_users(filepath):  
    with open(filepath, "r") as f:  
        data = json.load(f)  
  
    users = []  
    for item in data:  
        if item.get("role") == "admin":  
            # Instantiate Admin object  
            users.append(Admin(item["user_id"], item["name"]))  
        else:  
            # Instantiate User object  
            users.append(User(item["user_id"], item["name"]))  
  
    return users
```

Core Logic & Speed (Sultanali)

O(1) Access & Guard Clauses

- ⚡ **O(1) Dictionaries:** By structuring data as {id: object} in dictionaries, lookups are instant.
- 🛡️ **Guard Clauses:** Transactions validate all conditions early, returning False to prevent failures.

```
def borrow_book(self, user_id, book_id):  
    user = self.find_user(user_id)  
    book = self.find_book(book_id)  
  
    # Guard clauses (Early Returns)  
    if user is None or book is None:  
        return False  
  
    if not book.is_available():  
        return False  
  
    # Safe state mutation  
    book.set_available(False)  
    user.borrow_book(book_id)  
  
    return True
```

Interface & Testing (Rizabek)

QA & Reliability

- 🎮 **The Controller:** main.py completely decouples UI from the Library's backend.
- 🔧 **Unit Tests:** 15+ automated tests verify edge cases.

```
class TestUser(unittest.TestCase):  
  
    def test_borrow_same_book_twice(self):  
        user = User(1, "Alice", [101])  
        with self.assertRaises(ValueError):  
            user.borrow_book(101)  
  
    def test_return_book_not_borrowed(self):  
        user = User(1, "Alice")  
        with self.assertRaises(ValueError):  
            user.return_book(999)
```